TECHNICAL DOCUMENTATION

Medical Data Lakehouse Platform

From raw clinical records to privacy-safe, analytics-ready data

FHIR and DICOM ingestion | De-identification | Data quality gates

Spark | Apache Iceberg | Project Nessie | Trino | Airflow

Docker | Kubernetes | Helm | Argo CD

Document

Version	1.0
Date	2 October 2026
Scope	Architecture, implementation, operations, impact and limitations
Audience	Engineers, DevOps, data stewards, technical reviewers

Contents

1. Executive summary . . . 4

2. The problem this project addresses . . . 5

2.1 Clinical data is valuable, messy and sensitive . . . 5
2.2 Why ad-hoc scripts fall short . . . 5
2.3 What success looks like . . . 5

3. Solution overview . . . 6

3.1 End-to-end journey of a single record . . . 6

4. System architecture . . . 7

4.1 Components and the reasoning behind them . . . 7
4.2 A deliberate design principle: thin orchestration, thick libraries . . . 8
4.3 One codebase, three run modes . . . 8

5. Data flow in detail . . . 9

5.1 Stage 1 - Ingestion and validation . . . 9
5.2 Stage 2 - Transformation from Bronze to Silver . . . 10
5.3 Stage 3 - Aggregation from Silver to Gold . . . 10
5.4 Stage 4 - Serving . . . 10

6. Data model . . . 11

6.1 Silver tables . . . 11
6.2 Gold marts . . . 12

7. Privacy and de-identification . . . 13

7.1 The field-by-field policy . . . 13
7.2 What the transformation looks like on real output . . . 13
7.3 Design decisions worth understanding . . . 14
7.4 What this does and does not achieve . . . 14

8. Data quality and reliability . . . 16

8.1 The checks . . . 16
8.2 Quarantine: nothing is silently dropped . . . 16
8.3 What happened on the reference run . . . 16
8.4 Reliability features . . . 17

9. Orchestration . . . 18

10. Infrastructure and DevOps . . . 19

10.1 Container image . . . 19
10.2 Docker Compose stack . . . 19
10.3 Kubernetes manifests . . . 19
10.4 Helm chart for Garage . . . 20
10.5 GitOps with Argo CD . . . 20
10.6 Configuration and secrets . . . 21
10.7 Backup and recovery . . . 21
10.8 Python and Java compatibility . . . 21

11. Testing and verification . . . 22

11.1 The test suite . . . 22
11.2 What was independently re-run for this document . . . 22

11.3 What has not been verified . . . 23

12. Engineering journey: defects found and fixed . . . 24

12.1 The most instructive defects . . . 24
12.2 The lesson behind the list . . . 24

13. Impact . . . 26

13.1 Impact on analytics and decision-making . . . 26
13.2 Impact on risk, privacy and governance . . . 26
13.3 Impact on engineering and operations . . . 27
13.4 What is demonstrated, and what still needs measuring . . . 27
13.5 Practices this project demonstrates . . . 27

14. Limitations and production readiness . . . 28

14.1 Deployment and verification gaps . . . 28
14.2 Scale and architecture . . . 28
14.3 Operations and security . . . 29

15. Roadmap . . . 30

Phase 1 - Prove it on real infrastructure . . . 30
Phase 2 - Make it production-grade . . . 30
Phase 3 - Extend the platform . . . 30

Appendix A. Operations runbook . . . 31

A.1 Run the pipeline locally . . . 31
A.2 Investigate a quarantined record . . . 31
A.3 Docker Compose . . . 31
A.4 Kubernetes . . . 31
A.5 Troubleshooting . . . 32

Appendix B. Configuration reference . . . 33

Appendix C. Repository map . . . 34

Appendix D. Glossary . . . 35

How to read this document

Chapters 2-5 explain the *what* and *why* and suit every reader. Chapters 6-9 go into the data model, privacy design, quality controls and orchestration. Chapter 10 is written for DevOps and platform engineers. Chapters 11-13 cover verification, engineering history and impact. Chapter 14 sets out honest limitations and Chapter 15 the roadmap. The appendices hold the operations runbook, configuration reference, repository map and glossary.

1. Executive summary

Hospitals and health systems sit on enormous amounts of clinical data, but most of it is locked inside operational systems that were never designed for analysis. The records that would let an organisation understand its patient population, its utilisation of care, or the quality of its services are scattered across feeds and files, inconsistent in quality, and full of information that identifies real people.

The **Medical Data Lakehouse Platform** is an end-to-end data engineering system that closes that gap. It takes clinical data in the two formats healthcare actually uses — **HL7 FHIR** resources (patients, encounters, observations, conditions and medication orders) and **DICOM** imaging metadata — and turns it into governed, de-identified, analytics-ready tables that analysts can query with ordinary SQL. Every step in between is automated, validated, logged and tested.

The platform is built around four commitments, and each one is enforced by code rather than by convention:

- **Privacy first.** Direct identifiers (names, SSN, phone, email, address) are removed before analytics data is written. The medical record number is replaced by a salted hash, birth dates become a birth year and an age band, and postal codes are cut to a three-digit prefix. A final guard refuses to write any table that still contains a forbidden field.
- **Nothing is silently lost.** Records that fail validation or break a relationship rule are written to a quarantine area together with the exact reason, so data stewards can inspect and fix them instead of wondering where rows went.
- **Bad batches cannot reach analysts.** A configurable quality gate stops the pipeline when too large a share of a batch is invalid, before anything is published downstream.
- **Reproducible and portable.** The same code runs on a laptop with no infrastructure, in Docker Compose, and on Kubernetes under a GitOps workflow managed by Argo CD.

80	6	5	7
automated tests (74 pass standalone, 1 needs Airflow)	Silver tables (5 FHIR + DICOM metadata)	Gold analytics tables	orchestrated pipeline stages

This document explains the platform in full: the problem it addresses, how it is designed and why each technology was chosen, how data moves and is protected, how the system is deployed and operated, what was verified, and — just as importantly — where the boundaries of the current implementation lie. The authors chose to describe the system as it actually is. Where a capability is proven by tests or by an end-to-end run, this document says so; where it is designed but not yet exercised against live infrastructure, it says that too (see Chapters 11 and 14).

2. The problem this project addresses

2.1 Clinical data is valuable, messy and sensitive

A modern hospital generates a continuous stream of structured records: a patient is registered, an encounter opens, observations such as blood pressure and glucose are captured, a clinician records a diagnosis, a medication is ordered, and an imaging study is performed. Each of these is stored by a different operational system, usually optimised for treating the individual patient in front of the clinician — not for answering population-level questions.

Leadership, quality teams and researchers want precisely those population-level answers. How many patients do we see in each age group? What share of encounters are emergency visits? Which conditions are most common, and how many patients are on five or more medications at once? Answering these reliably requires data that is consistent, linked correctly across record types, and safe to share with analysts.

2.2 Why ad-hoc scripts fall short

The common first attempt is a collection of one-off scripts that export, clean and aggregate data. This approach breaks down in predictable ways, each of which this platform was designed to prevent:

Typical failure	What goes wrong	How the platform responds
PHI leaks into analytics	A name, SSN or exact birth date ends up in a table that many people can query.	De-identification happens before Silver is written; a forbidden-field check blocks the write if anything slips through.
Silent data loss	Malformed or orphaned rows are dropped and nobody notices the totals are wrong.	Every rejected record is persisted to quarantine with its reason; row counts are tracked in structured metrics.
Broken relationships	An observation refers to a patient or encounter that does not exist, so joins quietly lose rows.	Referential integrity is checked in two passes and violating rows are quarantined, not discarded.
Unrepeatable runs	Re-running a job duplicates data or overwrites earlier work.	Silver uses create-then-MERGE by key in cluster mode; reruns update rows in place.
No audit trail	Nobody can say which file or batch a number came from.	Every record carries batch_id, source_file and ingestion timestamp; Bronze keeps the raw original.
Environment drift	It works on one machine and nowhere else.	Environment-variable configuration, containers, Helm charts and GitOps keep environments declarative.

2.3 What success looks like

For this project, success was defined as a pipeline that a careful reviewer could trust: it ingests real FHIR and DICOM-metadata structures, protects the individuals in the data, refuses to publish bad data, leaves a clear trail for every decision, and can be deployed with the same discipline used for production software.

3. Solution overview

The platform follows the **medallion architecture**, a widely used lakehouse pattern in which data is refined through successive layers. Each layer has a clear purpose, and data moves in one direction only.

Layer	Purpose	Contents	Who may see it
Bronze	Faithful, replayable record of what arrived	Raw FHIR resources and DICOM metadata exactly as received, plus batch_id, source_file and ingestion timestamp.	Tightly restricted. Contains raw PHI by design, for auditability.
Quarantine	Evidence of everything that was rejected	Rejected records with error type, message, source and the original payload.	Data stewards and engineers.
Silver	Clean, typed, de-identified, linked tables	patient, encounter, observation, condition, medication_request, dicom_metadata.	Analysts and data scientists (further restrictable through views).
Gold	Business-ready aggregates	patient_summary, observation_daily_summary, encounter_summary, condition_summary, medication_summary.	Dashboards, BI tools, broad reporting audiences.

In plain terms: data lands in **Bronze** untouched; anything unacceptable is diverted to **Quarantine**; the rest is cleaned, de-identified and linked into **Silver**; and Silver is summarised into **Gold** tables shaped like the questions people actually ask. Analysts connect to Trino and write SQL against Silver and Gold.

3.1 End-to-end journey of a single record

It helps to follow one record through the system. Take the sample patient *pat-1001* from the bundled test data:

1. The FHIR bundle file is read. The patient resource passes validation (it has an id, and its birth date parses), so it is written to Bronze with a batch id and timestamp. The raw record still contains the patient's name, SSN, phone, email and street address.
2. The Spark job parses the nested FHIR JSON into a flat row, then de-identifies it. The name, SSN, phone, email and address are dropped. The MRN becomes a 64-character hash. The birth date 1982-04-17 becomes birth year 1982 and age band 40-49. The ZIP code 62704 becomes the prefix 627.
3. The patient's encounter, observation, condition and medication records are checked: does each one point at a patient that exists? At an encounter that exists? All do, so they proceed.
4. A PHI-leak assertion confirms no forbidden column is present, and the Silver tables are written.
5. The Gold job counts that patient's encounters, conditions and medications into patient_summary, and contributes to the daily observation, encounter, condition and medication summaries.
6. An analyst can now ask how many 40-49 year-olds have a diabetes diagnosis — without ever being able to learn who any of them are.

4. System architecture

The diagram below shows the logical components and the direction of data flow. Apache Airflow sits across the top and orchestrates every stage; the numbered boxes show where data is held and transformed; the band at the bottom shows the deployment tooling.

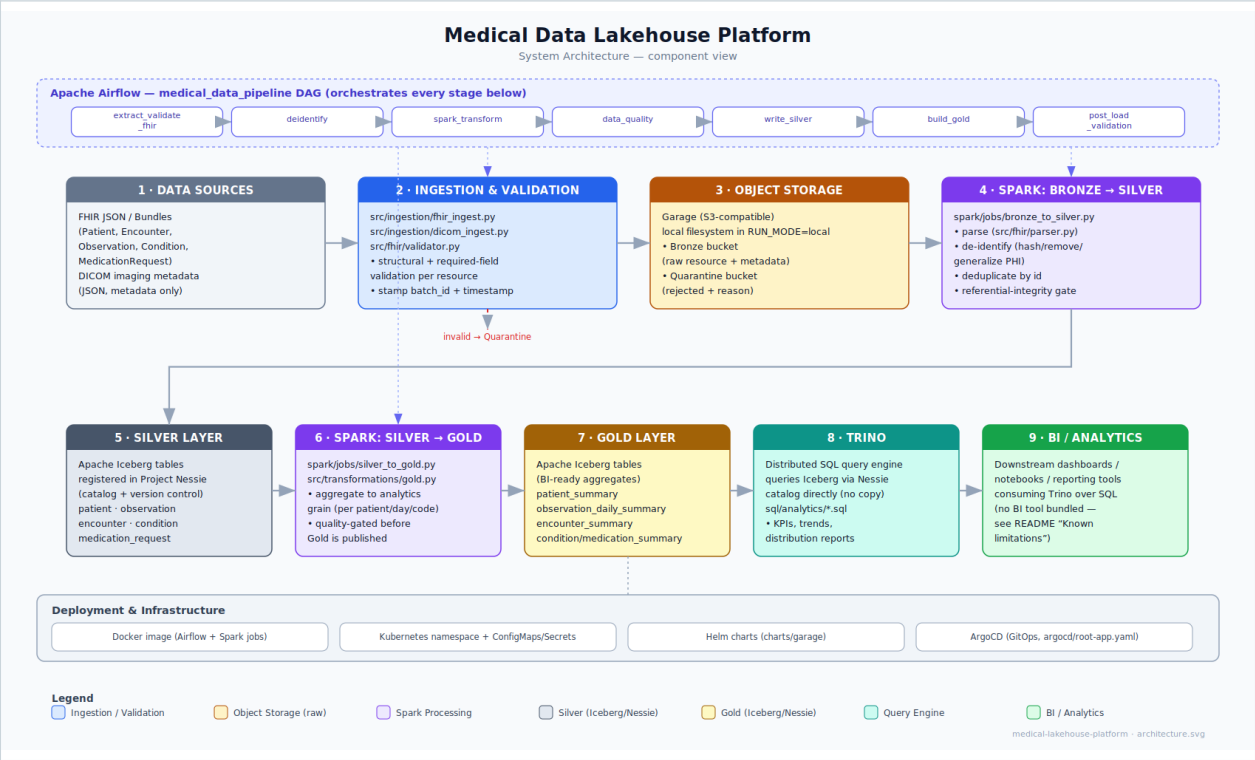


Figure 1 - System architecture (component view).

4.1 Components and the reasoning behind them

Technology choices were made for specific reasons rather than fashion. The table records each choice and the problem it solves.

Layer	Technology	Why it was chosen
Orchestration	Apache Airflow	Explicit task dependencies, automatic retries, scheduling, and a UI where operators can see and rerun failed stages.
Processing	Apache Spark (PySpark)	The same job code runs locally in local[*] mode and on a cluster, so development and production share logic.
Table format	Apache Iceberg	ACID writes, schema evolution, hidden partitioning and snapshots on top of plain Parquet files in object storage.
Catalog	Project Nessie	Git-like versioning of table state (branches, commits, tags) for reproducibility and safe experimentation.
Object storage	Garage (S3-compatible)	Lightweight, self-hosted S3 that fits Kubernetes and avoids vendor lock-in; any S3 endpoint can replace it.
Query engine	Trino	Interactive SQL directly over Iceberg tables without copying data into a separate warehouse.

Layer	Technology	Why it was chosen
Packaging	Docker, Docker Compose	One image carries the pipeline code, a JDK and dependencies; Compose gives a local multi-service stack.
Deployment	Kubernetes, Helm, Argo CD	Declarative, repeatable deployment; Git is the single source of truth and Argo CD reconciles the cluster to it.
Quality	pytest, ruff	80 automated tests and a clean static-analysis pass guard against regressions.

4.2 A deliberate design principle: thin orchestration, thick libraries

The Airflow DAG contains almost no business logic. Each task calls into reusable, independently tested Python modules under `src/`, or launches a Spark entry point under `spark/jobs/`. This has three practical consequences. First, the pipeline can run completely without Airflow (the script `scripts/run_pipeline_local.sh` does exactly that). Second, every rule — validation, de-identification, quality checks — can be unit tested in isolation in milliseconds. Third, replacing the orchestrator in the future would not require rewriting any business rule.

4.3 One codebase, three run modes

Mode	Storage	Spark	Silver / Gold output	Best for
Local (RUN_MODE =local, default)	Local filesystem under data/	local[*] on one machine	Plain Parquet, one folder per batch	Development, CI, demos. Needs only Python and a JDK.
Docker Compose	Garage, Postgres, Nessie, Trino containers alongside Airflow	local[*] inside the Airflow image	Parquet on a mounted volume (Airflow runs in local mode in this stack)	Trying the services together on one machine.
Cluster (RUN_MODE=cluster)	S3-compatible object storage	Spark with Iceberg and Nessie jars	Iceberg tables in Nessie: iceberg.medical.* and iceberg.medical_gold.*	Kubernetes deployment under Argo CD.

> **Why this matters**
> A single configuration switch moves the same code between a laptop and a cluster. The trade-off is that local mode writes Parquet rather than real Iceberg tables, so multi-batch accumulation and MERGE behaviour can only be reasoned about in cluster mode (see Chapter 14).

5. Data flow in detail

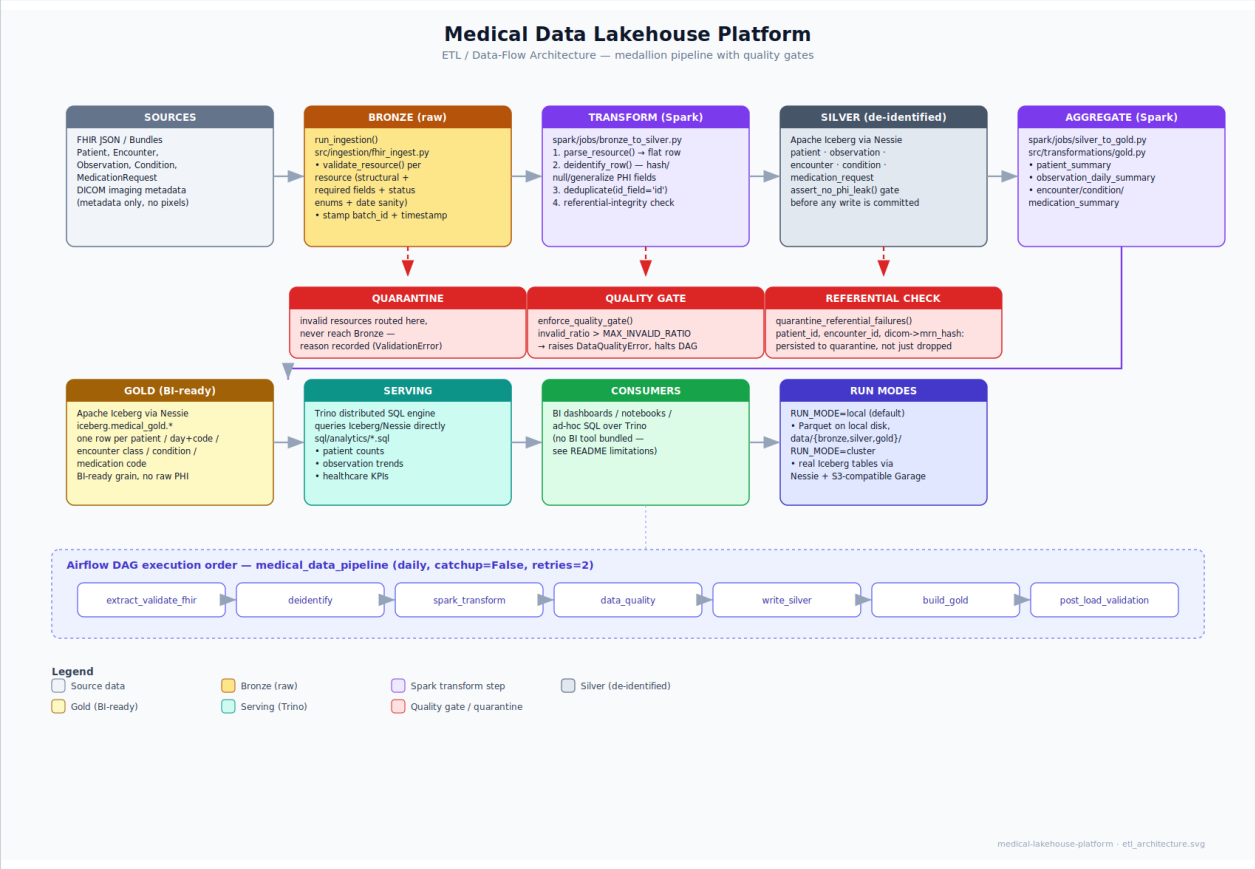


Figure 2 - ETL and data-flow architecture, including the quality-gate and quarantine branches.

5.1 Stage 1 - Ingestion and validation

Ingestion reads FHIR JSON files from a source directory. A file may hold a single resource or a FHIR *Bundle* with many entries; both are handled. Each resource is validated individually, so one bad record never blocks the good ones around it.

The validator is intentionally pragmatic. It checks that the resource type is supported (Patient, Observation, Encounter, Condition, MedicationRequest), that the required fields for that type are present, and a small set of value rules: dates must parse, and status fields must hold legal FHIR codes. It is not a full FHIR profile validator, and the documentation says so plainly.

Resource	Required fields	Extra rules
Patient	id	birthDate must parse as a date.
Observation	id, status, code	status in the FHIR set; valueQuantity, if present, must carry a numeric value.
Encounter	id, status	status in the FHIR set.
Condition	id, subject	-
MedicationRequest	id, status, subject	status in the FHIR set.

Valid resources are stamped with `ingestion_timestamp`, `source_file` and `batch_id` and written to Bronze. Invalid resources are written to quarantine with the specific validation error. Unreadable files are quarantined too, as a *FileReadError*.

DICOM metadata follows the same pattern in the same Airflow task, validated against a combined invalid-ratio threshold. The platform processes DICOM *metadata* — patient, study and series attributes — not the image pixels; full image processing is out of scope. In production the metadata would be extracted from `.dcm` headers by a library such as pydicom; the bundled fixture is already-extracted JSON.

5.2 Stage 2 - Transformation from Bronze to Silver

The Spark job `bronze_to_silver.py` performs the central work of the platform in a deliberate order:

1. **Parse** each nested FHIR resource (with its CodeableConcepts, References and Periods) into a flat, typed row.
2. **De-identify** the row according to the policy in Chapter 7.
3. **Deduplicate** by natural key (the resource id, or the SOP Instance UID for DICOM).
4. **Check references, pass 1:** every *patient_id* must exist in the patient table; for DICOM, the hashed patient id must match a patient's *mrn_hash*.
5. **Check references, pass 2:** every *encounter_id* must exist among the encounters that survived pass 1.
6. **Quarantine** every violating row as a full record, with the foreign key that failed and why.
7. **Apply the quality gate** to the quarantined share, *before* anything is written to Silver.
8. **Assert no PHI** can reach Silver, then write the tables.

The ordering is deliberate. Because the quality gate runs before the Silver write, a batch that fails the gate leaves no partial Silver output behind. Because the second reference check runs against the encounters that *survived* the first, a child record can never point at an encounter that was itself quarantined.

5.3 Stage 3 - Aggregation from Silver to Gold

The Gold job reads Silver, computes the five summary tables, and writes them with explicit schemas. Gold is always a **full recompute** from Silver rather than an increment, which is why replacing the table on each run is correct and idempotent. One rule is enforced throughout: **every Silver row must land in exactly one Gold group**. All group-by operations keep rows whose grouping key is null, because the default behaviour of the underlying library is to drop them silently — a defect that was found during review and is now covered by regression tests (Chapter 12).

5.4 Stage 4 - Serving

Trino queries Iceberg tables through the Nessie catalog, reading the Parquet files in object storage directly. Six groups of example queries live in `sql/analytics/`: patient counts, observation trends, encounter statistics, condition distribution, medication statistics and a small set of healthcare KPIs. No BI tool is bundled; any tool that speaks SQL to Trino can sit on top.

6. Data model

The Silver layer is a small, conventional relational model held in Iceberg tables. Patients sit at the centre; encounters, observations, conditions and medication requests all refer back to a patient, and most also refer to the encounter during which they occurred. DICOM imaging metadata links to the patient through the hashed medical record number.

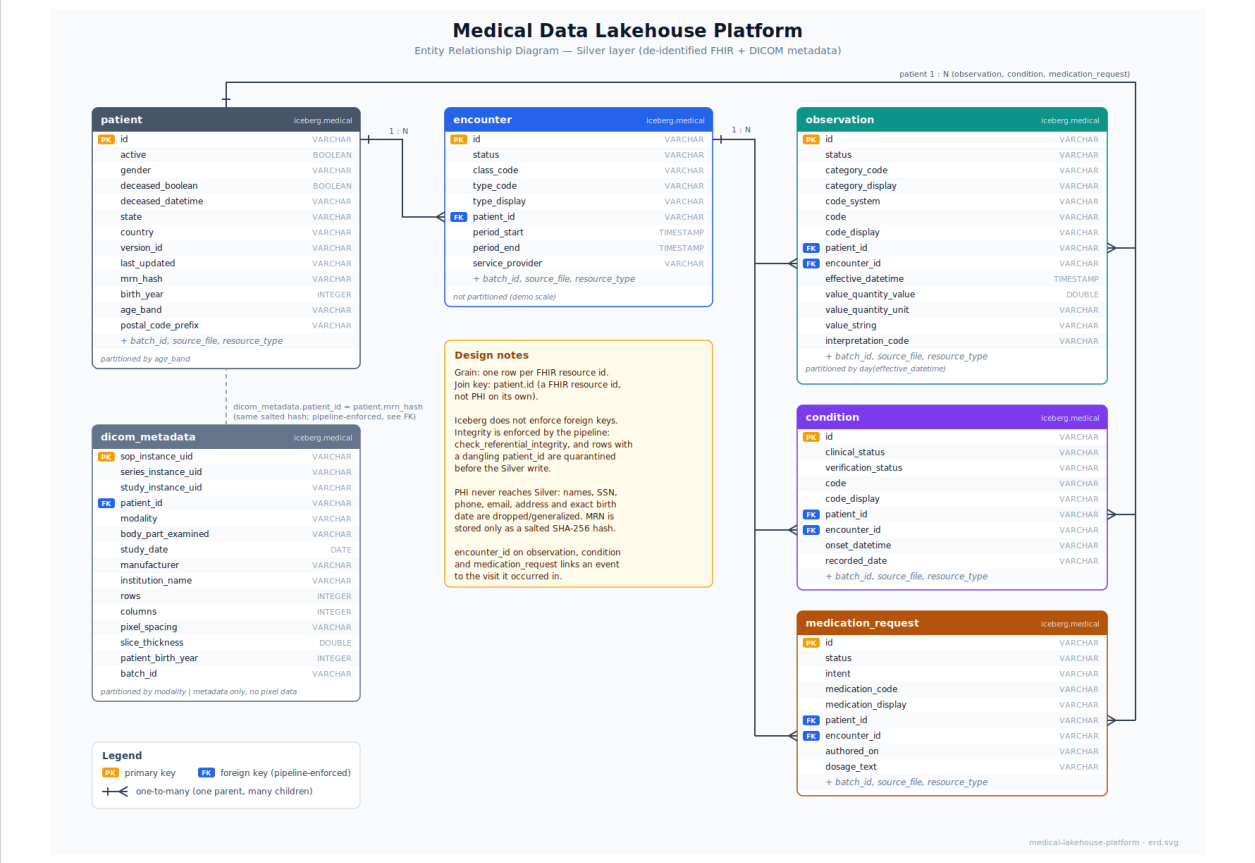


Figure 3 - Entity-relationship diagram for the Silver layer.

6.1 Silver tables

Table	Natural key	References	Partitioning (cluster mode)
`patient`	`id`	-	age_band (a handful of coarse buckets)
`encounter`	`id`	patient_id -> patient.id	none at current volume
`observation`	`id`	patient_id, encounter_id	day(effective_datetime)
`condition`	`id`	patient_id, encounter_id	none at current volume
`medication_request`	`id`	patient_id, encounter_id	none at current volume
`dicom_metadata`	`sop_instance_uid`	patient_id -> patient.mrn_hash	modality

Iceberg does not enforce foreign keys, so the pipeline enforces them itself (Chapter 8). Two design decisions in the partitioning are worth noting. The patient table is partitioned by age band, *not* by gender as an early draft had it, because gender has too few distinct values to prune any meaningful amount of data. The observation

table is partitioned by day of the measurement because it is event data and almost every query filters on a date range.

One more schema decision came from a defect: observation codes are stored as a *code_system* plus a *code*, never as a single LOINC-only column. An earlier version labelled every code as LOINC regardless of its real coding system, and later nulled non-LOINC codes. Storing the system and code together means the code is never discarded or mislabelled.

6.2 Gold marts

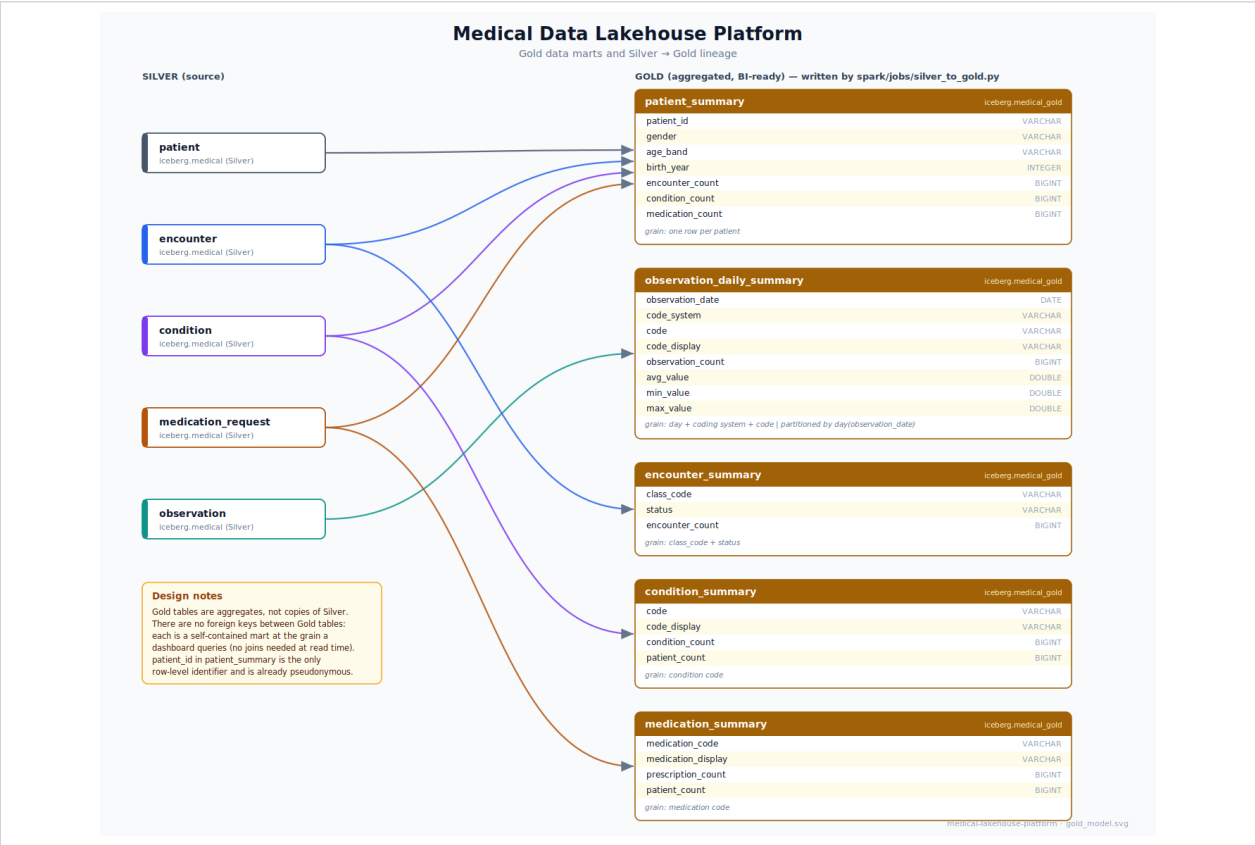


Figure 4 - Gold data marts and their lineage from Silver.

Gold table	Grain (one row per…)	Main measures
`patient_summary`	patient	encounter_count, condition_count, medication_count, with gender, age_band, birth_year
`observation_daily_summary`	day + code system + code	observation_count, avg_value, min_value, max_value
`encounter_summary`	encounter class + status	encounter_count
`condition_summary`	condition code	condition_count, patient_count (distinct patients)
`medication_summary`	medication code	prescription_count, patient_count (distinct patients)

Gold tables are deliberately not copies of Silver. Each is aggregated to the grain a dashboard would actually query, which keeps dashboards fast and keeps the underlying patient-level rows out of broad reporting views.

7. Privacy and de-identification

Protecting the individuals in the data is the most important function of the platform, so it is implemented as testable code with a written policy rather than as a convention that people are expected to remember.

7.1 The field-by-field policy

Source field	Classification	Treatment in Silver
`mrn`	Direct identifier	Replaced by a deterministic, salted SHA-256 hash stored as mrn_hash.
`ssn`	Direct identifier	Removed.
`first_name, last_name`	Direct identifier	Removed.
`phone, email`	Direct identifier	Removed.
`address_line, city`	Direct identifier	Removed.
`postal_code`	Quasi-identifier	Generalised to the first three characters (postal_code_prefix).
`birth_date`	Quasi-identifier	Generalised to birth_year and a ten-year age_band; ages 90 and over share one band.
`gender, state, country`	Low risk alone	Kept.
`id (FHIR resource id)`	Opaque system key	Kept unchanged so that joins between tables keep working.
`DICOM patient_name`	Direct identifier	Removed.
`DICOM patient_id`	Direct identifier (holds the MRN)	Hashed with the same function as mrn_hash.
`DICOM patient_birth_date`	Quasi-identifier	Reduced to patient_birth_year.

7.2 What the transformation looks like on real output

The table below compares the synthetic sample patient *pat-1001* as it sits in Bronze with the same patient in Silver. The Silver values were produced by running the pipeline on the bundled sample data (with the local development salt, so the hash value will differ in any real deployment).

Attribute	Bronze (raw)	Silver (de-identified)
Name	Elena Rivera	removed
MRN	MRN-778812	78f35b11…3a36c8b4…d6907f (SHA-256, shortened here)
SSN	123-45-6789	removed
Phone / email	555-201-8834 / elena.rivera@example.com	removed
Address	482 Maple Street, Springfield, IL 62704	state = IL; postal_code_prefix = 627
Birth date	1982-04-17	birth_year = 1982; age_band = 40-49
Resource id	pat-1001	pat-1001 (unchanged, used as join key)

A search of the generated Silver output for the sample SSN returned nothing, and the Silver patient schema contains none of the removed columns.

7.3 Design decisions worth understanding

Deterministic hashing. The same MRN always produces the same hash. That is a deliberate trade-off: it allows imaging metadata and patient records to be linked, and lets analysts count distinct patients, without ever seeing the MRN. A random token would be stronger in isolation but would break those joins.

Why the FHIR patient id is not hashed. References on observations, encounters, conditions and medication requests point to the patient's FHIR resource id, which is an opaque key assigned by the source system rather than a recognised identifier. Hashing one side of a foreign key and not the other silently breaks every join. This exact mistake occurred during development and was caught by the referential-integrity check, which is precisely the job that check exists to do.

The DICOM link. DICOM stores the patient's MRN in the PatientID tag (0010,0020). Hashing it with the same salted function produces the same value as the patient table's mrn_hash, so dicom_metadata.patient_id = patient.mrn_hash. An earlier version documented this link but populated the field from a different source, so the two never matched; it was fixed at the data level and is now enforced and tested. The link holds only where a site's DICOM PatientID really is the MRN.

Secrets stay out of the repository. The hashing salt is read from the DEID_SALT environment variable or a Kubernetes Secret and is never hardcoded in a deployment file. Docker Compose refuses to start unless it is set.

A last line of defence. Before every Silver write, `assert_no_phi_leak` checks every table against a list of forbidden fields (ssn, names, phone, email, address, city, raw MRN, exact birth_date) and raises an error if any is present. De-identification therefore has to fail twice before PHI could be written.

7.4 What this does and does not achieve

> **An honest statement of privacy posture**
> The platform is designed with HIPAA-oriented controls in mind and applies Safe-Harbor-style generalisation, but it is **not HIPAA compliant**, and no software project is by itself; compliance also needs signed agreements, access governance, physical and organisational controls, training and audits. Technically, the Silver layer is best described as **pseudonymised** data suited to controlled-access analytics, rather than fully anonymised data suited to public release.

Several details explain that distinction and are worth a reviewer's attention before any production use:

- **Event dates are retained.** Observation timestamps, encounter periods and DICOM study dates keep their full precision in Silver. They are essential for trend analysis, but the formal Safe Harbor method asks for date elements other than the year to be removed.
- **The MRN hash is a stable pseudonym.** Safe Harbor does not allow a code derived from an individual's own identifier. The hash is the right tool for linkage inside a controlled environment, but it is still a persistent identifier.
- **A single salted SHA-256 has limits.** Medical record numbers come from a small, structured space, so if the salt were ever exposed they could be recovered by guessing. A keyed HMAC with the key held in a managed secret store, and a rotation plan, would be a stronger design.
- **The three-digit ZIP prefix** is only acceptable where the area it covers is sufficiently populous; sparsely populated prefixes need special handling.
- **Age bands depend on the run date.** They are computed against today's date at processing time, so a patient near a decade boundary can move band between runs.
- **The development salt fallback.** If DEID_SALT is unset the configuration falls back to a clearly named development value. Compose requires the variable, but a cluster deployment should fail fast rather than fall back.

Bronze deliberately keeps the raw record, including all PHI, so that the pipeline is auditable and reproducible. It must therefore be access-controlled much more tightly than Silver and Gold. An example restricted Trino view (`v_patient_research`) shows how Silver can be narrowed further for a broad research audience by excluding even the MRN hash and birth year. A real audit trail would use Trino's event listener; that is documented in the SQL file but not implemented.

8. Data quality and reliability

8.1 The checks

Every check returns structured findings — row index, record id, field and message — rather than a bare true/false or a string to be re-parsed. That is what allows the pipeline to act on one specific row. (An early design matched rejected records by parsing identifiers out of error text, and silently let some bad records through. Structured findings removed the whole class of bug.)

Check	What it enforces	On failure
Required fields	Mandatory fields are present and non-empty for each resource type.	Record quarantined at ingestion.
Value rules	Status codes are legal FHIR values; dates parse; quantities carry a numeric value.	Record quarantined at ingestion.
Duplicate detection	One row per natural key; later duplicates are removed and counted.	Duplicate dropped, counted in metrics.
Referential integrity	patient_id and encounter_id exist in their parent tables; DICOM patient_id matches a patient mrn_hash.	Row quarantined as a full record with the failing key.
Observation value sanity	A finalised observation must carry a value.	Reported as a quality finding.
Forbidden fields	No raw PHI column exists in any Silver table.	Run aborts before write.
Quality gate	Invalid share of the batch stays at or below MAX_INVALID_RATIO (default 20%).	DataQualityError fails the Airflow task.

8.2 Quarantine: nothing is silently dropped

All rejection paths — FHIR validation, DICOM validation and referential-integrity failures — write through one helper, so every quarantined record has the same shape on disk: resource type, record id, error type, error message, source file, ingestion timestamp, batch id and the original payload. Someone investigating a rejection does not need to know which stage produced it.

8.3 What happened on the reference run

To ground these ideas, the pipeline was run end to end on the bundled sample data. The ingestion step read four FHIR files containing 18 resources and two DICOM metadata records.

Stage	Result
FHIR ingestion	18 resources read: 15 accepted into Bronze, 3 rejected to quarantine. Invalid ratio 16.7%, under the 20% gate, so the batch continues.
DICOM ingestion	2 records read, 2 accepted, 0 rejected.
Bronze to Silver	17 rows written: patient 3, observation 4, encounter 3, condition 3, medication_request 2, dicom_metadata 2. 0 referential quarantines, 0 duplicates.
Silver to Gold	5 Gold tables written: patient_summary 3 rows, observation_daily_summary 4, encounter_summary 2, condition_summary 3, medication_summary 2.
Join check	Every DICOM record's patient_id matched a patient mrn_hash.

The three rejected records come from a deliberately broken sample file, and each shows a different failure mode with a precise reason:

Record	Reason recorded in quarantine
Observation obs-9001	Invalid Observation.status 'not-a-real-status'.
Patient (no id)	Missing required field 'id'; invalid birthDate 'not-a-date'.
LabReport unsupported-1	Unsupported resourceType 'LabReport'; supported types are listed in the message.

This is the behaviour the platform promises: the bad rows are caught and explained, and the good rows in the same batch continue to Silver and Gold unharmed.

8.4 Reliability features

- **Retries and timeouts.** Every Airflow task retries twice with a five-minute delay and has a one-hour execution timeout.
- **One run at a time.** `max_active_runs=1` and `catchup=False` prevent overlapping runs and accidental backfill storms.
- **A gate must inspect something.** The Silver quality task fails if it finds zero records. An earlier version looked for JSON files in a folder of Parquet files, saw nothing, and passed — a gate that checked nothing. It now fails loudly and has a test that proves it can detect each kind of problem.
- **Idempotent writes.** In cluster mode Silver tables are created on first write and then updated with MERGE INTO by key, so reruns update rows instead of duplicating or wiping them. Gold is a full recompute.
- **Structured logging.** Every log line is a JSON object carrying fields such as batch_id, table and row counts, ready to ship to any log aggregator without a custom parser.
- **Explicit schemas.** Spark schemas are declared for every table, because small batches often contain all-null columns that defeat schema inference.

9. Orchestration

The Airflow DAG `medical_data_pipeline` chains seven tasks in a straight line. Large data never travels through Airflow itself; tasks pass only a batch id and light metrics through XCom, while the data moves through Bronze, Silver and Gold storage.

#	Task	What it does	Fails when
1	`extract_validate_fhir`	Runs FHIR and DICOM ingestion, writes Bronze and quarantine, evaluates the combined gate.	Invalid ratio exceeds the threshold.
2	`deidentify`	Applies the de-identification functions to each Bronze record and stages the output, making the step visible at the orchestration level.	Bronze data cannot be read.
3	`spark_transform`	Runs bronze_to_silver.py: parse, de-identify, deduplicate, reference checks, gate, PHI assertion, write Silver.	The Spark job exits non-zero, or its internal gate trips.
4	`data_quality`	Runs the Silver quality checks over the written Parquet and enforces the gate.	Zero records inspected, or invalid ratio too high.
5	`write_silver`	Checkpoint confirming the core Silver tables (patient, observation) exist.	Expected tables are missing.
6	`build_gold`	Runs silver_to_gold.py to produce the five Gold tables.	The Spark job exits non-zero.
7	`post_load_validation`	Confirms the expected Gold tables exist for the batch.	Expected Gold tables are missing.

The schedule defaults to daily and is controlled by the `MEDICAL_PIPELINE_SCHEDULE` variable, so the same DAG can serve a manual demo or an hourly production-like run. Source directories come from environment variables as well. The DAG's structure — exactly seven tasks in this dependency order, each with retries configured — is asserted by a test that imports it with a real Airflow installation.

> **A note on the de-identification task**
> The authoritative, full-batch de-identification (all resource types, plus DICOM) happens inside the Spark transform using the same functions. The standalone `deidentify` task demonstrates and stages the step for the DAG view; it does not feed the Spark job. This is a presentation choice, not a duplication of logic.

10. Infrastructure and DevOps

The platform is packaged and deployed with the same discipline expected of production software: a single container image, declarative configuration, secrets kept out of Git, and a GitOps workflow in which the repository is the source of truth. This chapter describes each layer and is intended for platform and DevOps engineers.

10.1 Container image

One image, built from `docker/Dockerfile.airflow`, serves both Airflow and the Spark jobs. It starts from the official Apache Airflow 2.9.3 image on Python 3.11, installs a headless Java runtime (required by PySpark), installs the pipeline's Python requirements, and copies in `src/`, `spark/`, `dags/` and the sample data. Because Airflow and its constraint-pinned dependencies come from the base image, Airflow is deliberately not installed a second time. Reusing one image for both purposes avoids maintaining two near-identical images and keeps the DAG's code and the Spark job's code in lockstep.

10.2 Docker Compose stack

Service	Image	Port(s)	Role
`postgres`	postgres:16-alpine	-	Airflow metadata database, with a health check.
`airflow-init`	built from Dockerfile.airflow	-	Runs database migration and creates the admin user.
`airflow-webserver`	built from Dockerfile.airflow	8080	Airflow UI and health endpoint.
`airflow-scheduler`	built from Dockerfile.airflow	-	Schedules and runs the DAG (LocalExecutor).
`garage`	dxflrs/garage:v1.0.1	3900, 3901, 3903	S3 API, RPC and admin API.
`nessie`	ghcr.io/projectnessie/nessie:0.99.0	19120	Iceberg catalog, in-memory version store (demo only).
`trino`	trinodb/trino:451	8081	SQL query engine with the Iceberg catalog mounted read-only.

The Compose file is written to fail fast: the database password, Fernet key, admin password, de-identification salt and Garage secrets are all declared with the `${VAR:?message}` form, so the stack refuses to start with a clear message if any is missing. Two behaviours should be understood by anyone operating it. First, Nessie runs with an in-memory version store, so catalog state is lost on restart. Second, the Airflow containers run with `RUN_MODE=local`, so in this stack pipeline output lands as Parquet on the mounted data volume rather than being written into the Garage, Nessie and Trino services that run beside it. The stack is therefore best seen as a convenient local environment for the orchestration layer and the supporting services, not yet a fully wired lakehouse.

10.3 Kubernetes manifests

- **Namespace** `data-lake` holds every workload.

- **ConfigMap** `medical-pipeline-config` carries the non-sensitive settings: run mode, log level, quality threshold, S3 endpoint and region, bucket names, warehouse path and the Nessie connection details.
- **Secret template** `secret.example.yaml` documents the sensitive keys (DEID_SALT, S3 access and secret keys). Real values are applied directly with kubectl from a git-ignored `*.local.yaml` file and are never committed.
- **Spark job template** runs `bronze_to_silver.py` as a plain Kubernetes Job with a retry limit of two and resource requests of 500m CPU and 1 GiB memory (limits 2 CPU and 2 GiB). It reuses the Airflow image and loads configuration from the ConfigMap and Secret.

10.4 Helm chart for Garage

The repository ships its own Helm chart for Garage object storage under `charts/garage/`. It contains templates for the Garage deployment, a web UI deployment, a persistent volume claim, a ConfigMap, a Secret, a Service and an Ingress, with defaults in `values.yaml`. The RPC and admin secrets are supplied at install time (for example, generated with `openssl rand -hex 32`) rather than stored in the chart. The chart was reviewed by hand; running `helm lint` and `helm template` is a recommended first step before deploying.

10.5 GitOps with Argo CD

Deployment follows the **app-of-apps** pattern. A single root application watches the `argocd/apps/` directory in Git, and each file in that directory describes one component. When someone merges a change to Git, Argo CD notices and reconciles the cluster to match.

Application	Source	Notable configuration
`garage-storage`	charts/garage in this repository	Object storage in the data-lake namespace.
`nessie-catalog`	Upstream Nessie chart 0.95.0	Two replicas, JDBC version store backed by Postgres, credentials read from a pre-created Secret, S3 endpoint pointing at Garage.
`trino-query`	Upstream Trino chart 0.36.0	Two workers; Iceberg catalog with S3 keys injected from a Secret through Trino's environment-variable substitution so key material never appears in Git or in Helm values.
`airflow-orchestration`	Upstream Airflow chart 1.15.0	LocalExecutor, DAGs synchronised from Git, RUN_MODE=cluster, secrets attached via a Secret reference.

All applications use automated sync with **prune** and **selfHeal**. Prune removes resources that were deleted from Git; self-heal reverts manual changes made directly in the cluster. The practical benefits are that the cluster cannot drift quietly away from what the repository says, every change has an author and a review trail in Git history, and rolling back is a matter of reverting a commit.

> **Before applying to a cluster**
> The manifests contain placeholders that must be replaced: the Git repository URL (`<your-org>`) and the container registry for the Airflow image. The exact Helm value keys used to inject secrets into the Nessie and Trino charts vary between chart versions and should be confirmed against the pinned versions before the first sync.

10.6 Configuration and secrets

All configuration is read from environment variables by a single settings loader, so identical code runs unchanged from a laptop, through Compose, to Kubernetes. No secret has a real default. The table in Appendix B lists every variable. In short: non-sensitive values live in a ConfigMap or `.env`; sensitive values (the de-identification salt, S3 keys, Garage secrets and database passwords) live in Kubernetes Secrets or an untracked `.env` file, with `.env.example` documenting what is needed.

10.7 Backup and recovery

A backup script, `scripts/backup_iceberg_warehouse.sh`, copies the four Garage buckets (raw, bronze, quarantine and the Iceberg warehouse) to a local directory or second S3 target using the S3-compatible mode of the AWS CLI. It is candid about its scope. It is run on demand, not on a schedule. It does not back up Nessie's backing database or Airflow's metadata database, both of which would need standard PostgreSQL tooling in a real deployment — and losing the Nessie store without a backup is effectively losing the catalog even if every Parquet file survives. The recovery point and recovery time are therefore whatever the operator makes them; neither has been measured.

10.8 Python and Java compatibility

The archive's name refers to a Python 3.14 compatibility fix, and the history behind it is a useful operational lesson. PySpark 3.5 predates Python 3.14, and its pip metadata does not forbid installing on it, so an install can appear to succeed and then fail deep inside Spark's serialisation layer. Separately, on Python 3.12 and later the standard library module `distutils` no longer exists, yet PySpark 3.5's pandas conversion still imports it, which broke the Gold step. The fix pins `setuptools<81`, whose bundled shim restores the import.

Python version	Status
3.11	Known-good path and the version used in the container image.
3.12, 3.13	Supported once the setuptools pin is in place. Python 3.12.3 with PySpark 3.5.9 and Java 21 was used to verify this document.
3.14	Not recommended until the project is verified against PySpark 4.1 or later, which added official 3.14 support.

The local setup script looks for Python 3.11, 3.12 or 3.13 on the path, in that order, and builds the virtual environment from the first one it finds, instead of trusting whatever `python3` happens to be. If none is found it stops with instructions rather than building an environment that would fail later at the Spark step. A JDK (11 or 17 per the project; Java 21 also worked in verification) is required for the Spark jobs.

11. Testing and verification

11.1 The test suite

The repository contains 80 automated tests across eleven files, roughly 1,000 lines of test code against about 2,500 lines of pipeline code. They are written to prove behaviour, not to inflate coverage: several use fixtures deliberately engineered to *fail* a check, so a checker that silently inspects nothing cannot pass by accident.

Test file	Tests	What it protects
`test_quality.py`	12	Every quality check and the quality-gate threshold behaviour.
`test_deidentification.py`	11	Hashing, generalisation, absence of forbidden fields, and the FHIR and DICOM join-key regressions.
`test_silver_checks.py`	10	Silver-layer checks against real Parquet fixtures built to fail each check.
`test_dicom_ingest.py`	8	DICOM validation and ingestion, including the record that is missing its own key.
`test_fhir_parser.py`	8	Flattening of all five FHIR resource types.
`test_gold_transformations.py`	8	Gold aggregates, including that no rows vanish for null grouping keys.
`test_validator.py`	7	FHIR structural validation rules.
`test_dag_import.py`	6	DAG imports cleanly, has exactly seven tasks in order, retries on every task (needs Airflow).
`test_iceberg_io.py`	4	Create-then-merge write sequencing, with mocks.
`test_spark_jobs.py`	4	Real PySpark runs: orphan references, DICOM join, quality gate blocking Silver, idempotent reruns.
`test_ingestion.py`	2	Integration test of ingestion against the real sample data.

11.2 What was independently re-run for this document

Rather than repeat the project's own claims, the key ones were re-executed from the delivered archive in a clean environment (Python 3.12.3, PySpark 3.5.9, pandas 2.3.3, Java 21):

Claim	Result of re-running it	Status
80 tests, all passing (74 + 1 skipped without Airflow)	74 passed, 1 skipped in about 35 seconds. The skip is the DAG import module, which requires Airflow.	Confirmed
Static analysis clean	ruff check on src, dags, spark and tests: all checks passed.	Confirmed
End-to-end local pipeline works	Ingestion, DICOM, Bronze to Silver and Silver to Gold all completed; counts in section 8.3.	Confirmed
Invalid records are quarantined with reasons	3 of 18 resources quarantined, each with a specific message.	Confirmed
No raw PHI reaches Silver	Silver patient schema has no removed columns; sample SSN not found anywhere in Silver output.	Confirmed
DICOM links to patients via mrn_hash	All DICOM patient_id values matched a patient mrn_hash.	Confirmed

11.3 What has not been verified

Equally important is what these runs do not cover. The following are written carefully but have not been executed against live infrastructure in the environments used so far:

- Docker Compose bring-up of the full service stack.
- Kubernetes manifests, the Garage Helm chart (no `helm lint` run) and the Argo CD applications.
- The Iceberg `MERGE INTO` path against a live Nessie catalog; only its call sequencing is tested, with mocks.
- Trino executing the analytics SQL; the queries are written against the DDL but not run.
- A live Airflow run of the DAG in this verification. (The project's changelog records a real `airflow dags test` run during external review, and the DAG structure tests are part of the suite.)
- Performance or scale: all testing uses small sample data, and no benchmarks exist.

12. Engineering journey: defects found and fixed

The project's changelog is unusually candid, and it deserves a place in this document because it shows how the quality of the system was earned. Two rounds of review — one internal, one external and more adversarial, which re-ran the pipeline for real and checked every claim in the documentation and diagrams against the code — found thirteen defects. Several were serious, and a green test suite had not caught them. Each fix now has a regression test that fails if the defect returns.

12.1 The most instructive defects

What was wrong	Why it mattered	How it was fixed and guarded
The quality gate inspected nothing. The Silver check searched for JSON files in a folder of Parquet files.	The gate always saw zero records and silently passed. It looked like a control but provided none.	Reads Parquet, moved into a testable module, fails if zero records are found; fixtures built to fail each check.
DICOM was documented as part of the pipeline but was never called by it.	Diagrams and design text promised a capability the code did not deliver.	DICOM ingestion joined the same task and gate; Spark writes dicom_metadata; row count asserted in a test.
The DICOM join key was false: the two sides were hashed from different inputs and never matched.	Imaging data could not be linked to patients despite the ERD saying it could.	Fixture carries the MRN as real DICOM does; join tested; link now pipeline-enforced.
Rows failing referential checks were logged and discarded, not quarantined.	Contradicted the design promise that nothing is silently dropped.	Violating rows are persisted through the shared quarantine helper; test reads them back from disk.
Only patient_id was ever checked; encounter_id was not.	The ERD claimed enforced encounter references that did not exist.	Second reference pass against surviving encounters, with an orphan-encounter test case.
Observation codes were mislabelled as LOINC, then nulled for non-LOINC systems; pandas dropped null group keys.	Observations silently vanished from every Gold aggregate.	Schema now stores code_system and code; every group-by uses dropna=False; row-conservation tests.
Silver used createOrReplace in cluster mode.	Only the latest batch would survive; earlier data wiped on each run.	Create on first write, then MERGE INTO by key; Gold stays a full replace; idempotency test.
Invalid DICOM records were accepted because rejection matched on an id token parsed from an error string.	Bad data entered Bronze; a record missing its own key caused a crash later.	String parsing replaced everywhere by structured RowIssue findings.
Referential checks hashed one side of a join key (found in development).	Every join would silently have lost rows.	Patient id left unhashed on both sides; the integrity check proved its value by catching it.

Three smaller issues round out the list: a no-op copy-paste expression in DICOM ingestion, an architecture diagram that listed two container images where there is one, and cache directories committed to the repository.

12.2 The lesson behind the list

Green tests are not the same as correct behaviour

The recurring pattern was a control that looked present but did nothing: a gate that read the wrong file type, a check described in the design but absent from the code, a diagram promising a join that never matched. The team's response was to treat documentation claims as testable, to write checks that must be able to *fail*, and to attach a regression test to every fix. That practice is as valuable as any individual feature in the platform.

13. Impact

Impact is easy to overstate in project documentation, so this chapter separates three things: the value the platform is built to create, the behaviour that has actually been demonstrated, and the measurements that would be needed to prove the value in production. No production deployment exists yet, so no real-world efficiency figures are claimed here.

13.1 Impact on analytics and decision-making

The most direct benefit is that people who need answers can get them from clean, trustworthy tables using ordinary SQL, without waiting for an engineer to write a one-off extract and without handling identifiable records. The Gold layer is shaped around the questions a hospital actually asks, and the bundled queries already answer several of them:

Question	Answered by
How many patients do we have, and how are they spread by gender and age band?	`01_patient_counts.sql`
How are key measurements, such as blood pressure or glucose, trending day by day?	`02_observation_trends.sql`
How many encounters are there by class and status, and who are the heaviest users of care?	`03_encounter_statistics.sql`
Which conditions are most common, and how often do they co-occur?	`04_condition_distribution.sql`
Which medications are prescribed most, and where is there a polypharmacy signal?	`05_medication_statistics.sql`
What is our emergency-encounter rate, average conditions per patient, abnormal-observation rate, and number of patients on five or more medications?	`06_basic_healthcare_kpis.sql`

Because every number traces back through Silver to a batch, a source file and an ingestion time, a reader who doubts a figure can follow it to its origin rather than argue about it.

13.2 Impact on risk, privacy and governance

- **A smaller privacy surface.** Analysts work in Silver and Gold, where direct identifiers have been removed or generalised. The only layer holding raw PHI is Bronze, which can be locked down far more tightly than a general analytics environment.
- **Controls that are enforced, not requested.** De-identification and the forbidden-field assertion are code paths that run on every batch. They do not depend on an analyst remembering a policy.
- **A workflow for data stewards.** Quarantine turns data problems from invisible losses into a reviewable queue with a stated reason for each record, which is the starting point for fixing issues at the source system.
- **Protection from bad numbers.** The quality gate stops a badly broken batch from becoming a dashboard figure that someone acts on.
- **Auditability.** Bronze preserves the original, batch ids and timestamps identify provenance, and Nessie adds versioned table history.

13.3 Impact on engineering and operations

- **Confidence to change things.** With 80 tests and a thin DAG, a rule can be changed and verified in seconds without launching a cluster.
- **Reproducibility and portability.** One configuration-driven codebase runs on a laptop, in containers and on Kubernetes, which shortens onboarding and removes the "works on my machine" class of problem.
- **Operational transparency.** Retries, timeouts, structured JSON logs and a visible seven-stage DAG make failures diagnosable by someone who did not write the code.
- **Declarative deployment.** Git-driven deployment with self-healing reduces configuration drift and makes rollback a routine operation.
- **Vendor independence.** Open formats (Parquet, Iceberg) and an S3-compatible API mean storage and compute can each be replaced or scaled without rewriting the pipeline.

13.4 What is demonstrated, and what still needs measuring

Outcome	Evidence today	How to measure it in production
PHI does not reach analytics tables	Field-level tests, forbidden-field assertion, and a clean Silver output on the sample run.	Periodic scans of Silver and Gold for identifier patterns; count of PHI incidents (target: zero).
Bad data is caught, not lost	3 of 18 sample resources quarantined with specific reasons; orphan-reference tests.	Quarantine rate by source system and error type, trended over time; time to resolve a quarantined record.
Bad batches are stopped	Gate tests; Spark test showing the gate blocks Silver output.	Number of batches blocked, and how many were true positives.
Trustworthy, fresh analytics	Sample run produces all five Gold tables.	Data freshness (landing to Gold), batch success rate, and analyst-reported discrepancies.
Faster answers for analysts	Example queries exist and match the Gold grain.	Query latency in Trino and the time to answer a new question compared with the previous process.
Safer, faster change	Regression test attached to each defect; clean lint; thin DAG.	Change failure rate, lead time for change, and mean time to recover.
Scalability	Design supports it (Spark, Iceberg partitioning, object storage).	Load tests at realistic volumes; not yet performed.

13.5 Practices this project demonstrates

The platform sits where data engineering meets DevOps, and it demonstrates the practices of both: medallion modelling, FHIR and DICOM domain handling, privacy-by-design, data-quality gates, orchestration with retries and timeouts, infrastructure packaged as containers and Helm charts, GitOps delivery with Argo CD, environment-driven configuration, secrets hygiene, structured logging, and a test-driven response to defects. Just as notably, it demonstrates professional honesty: a limitations list that does not flatter the work and a changelog that records mistakes plainly.

14. Limitations and production readiness

This chapter is the honest counterpart to the rest of the document. It lists what stands between the current implementation and a production deployment, so that nobody has to discover these points the hard way. None of them diminish what has been built; they define the next phase of work.

14.1 Deployment and verification gaps

Area	Current state	What production needs
Infrastructure verification	Compose, Kubernetes manifests, the Garage chart and the Argo CD applications were authored carefully but not deployed end to end.	A real bring-up: docker compose up, helm lint and template, kubectl dry-run, then a smoke test through Argo CD.
Iceberg MERGE on a live catalog	Call sequencing is unit tested with mocks; the generated SQL has not run against a live Nessie catalog.	An integration test against a real Nessie and Iceberg environment, including a multi-batch rerun.
Local versus cluster behaviour	Local mode writes per-batch Parquet and does not accumulate; cluster mode accumulates through MERGE.	Treat only cluster mode as the reference for multi-batch semantics.
DAG in cluster mode	Several DAG tasks (deidentify, data_quality, write_silver, post_load_validation) read local filesystem paths, so as written they suit RUN_MODE=local. In cluster mode the data lives in Iceberg and object storage.	Make those tasks storage-agnostic, reading through Iceberg or Trino, and then run the DAG against a cluster.
Compose wiring	Airflow runs in local mode, so Garage, Nessie and Trino in the Compose stack do not receive pipeline output.	Run Airflow in cluster mode in Compose, or document it explicitly as an orchestration-only environment.
Placeholders and missing files	Repository URL and image registry are placeholders; the README references a CI workflow that is not in the delivered archive; the helm/ directory is empty.	Restore the CI workflow, fill in the placeholders, and remove or populate empty directories.

14.2 Scale and architecture

Area	Current state	What production needs
Where the transformation runs	Parsing, de-identification, deduplication and reference checks run as Python over in-memory lists on the Spark driver; Spark handles typed writes. Gold aggregates run in pandas after a Spark read, and existing keys are collected to the driver.	Re-express these steps as native Spark DataFrame or SQL operations (anti-joins instead of Python sets) so they scale with the cluster.
Distributed Spark	Jobs run in local[*] mode. A single-pod Job template exists; a multi-node Spark-on-Kubernetes deployment does not.	Spark Operator or spark-submit against Kubernetes, with executors sized to the data.
Availability	Single replica everywhere: Garage replication factor 1, one volume, Nessie in memory in Compose.	Multi-node Garage (replication factor 3), Postgres-backed Nessie and Airflow, and replicated services.
Volume	Tested on a handful of sample resources.	Load and soak tests at realistic volumes with agreed service levels.

14.3 Operations and security

Area	Current state	What production needs
Backup and recovery	Manual script covering object storage only; recovery objectives unmeasured.	Scheduled backups including Nessie and Airflow databases, and a tested restore drill.
Monitoring	Structured JSON logs are shippable, but no Prometheus or Grafana deployment is included.	Metrics, dashboards and alerts on batch success, quarantine rate and freshness.
Access control and audit	An example restricted view exists; the audit trail is documented, not implemented.	Trino access control, an event-listener audit sink, and role-based access per layer.
Secret scanning	A pattern-based search in CI; no dedicated scanner or full history scan.	gitleaks or trufflehog in CI plus a history scan; fail fast when DEID_SALT is missing in cluster mode.
Privacy method	Safe-Harbor-style generalisation with retained event dates and a stable pseudonym (see section 7.4).	A privacy review, possibly Expert Determination, date shifting or keyed HMAC pseudonyms, and the organisational controls compliance requires.
FHIR validation	Pragmatic structural checks, not a profile validator.	Full profile validation, for example with the fhir.resources library.
Small checks	post_load_validation verifies four of the five Gold tables; medication_summary is not checked.	Add it to the expected list.

15. Roadmap

The following sequence turns the limitations above into an ordered plan. It starts with proving what already exists on real infrastructure, because every later improvement is more valuable on a verified foundation.

Phase 1 - Prove it on real infrastructure

1. Run `helm lint`, Compose bring-up and a Kubernetes dry-run; fix whatever they reveal.
2. Stand up Nessie, Garage and Trino together and execute the cluster-mode pipeline for several batches, verifying MERGE behaviour and the Trino queries.
3. Make the DAG's checkpoint tasks storage-agnostic so the whole DAG runs in cluster mode.
4. Restore the CI workflow, add secret scanning, and make a missing DEID_SALT a hard failure outside local mode.

Phase 2 - Make it production-grade

1. Back Nessie and Airflow with managed Postgres; schedule backups of every stateful component and rehearse a restore.
2. Move transformation steps to native Spark operations and deploy Spark on Kubernetes through the Spark Operator.
3. Add Prometheus and Grafana for batch success, quarantine rate, freshness and resource use; add alerting.
4. Deploy multi-node Garage with replication factor 3.
5. Add a BI layer such as Superset or Metabase on top of Trino for the Gold tables.

Phase 3 - Extend the platform

1. Strengthen privacy: keyed HMAC pseudonyms with managed keys and rotation, date shifting or an Expert Determination review, and fine-grained Trino access control with an audit sink.
2. Adopt full FHIR profile validation and data-contract tests for each source.
3. Add lineage with OpenLineage so every Gold column can be traced to its sources.
4. Introduce streaming ingestion (Kafka and Spark Structured Streaming) for near-real-time use cases, and process DICOM image data if the use case demands it.

Appendix A. Operations runbook

A.1 Run the pipeline locally

No Docker, Kubernetes, object storage or Spark cluster is needed. You need Python 3.11 to 3.13 and a JDK.

```
git clone <this-repo> && cd medical-lakehouse-platform
./scripts/init_local_env.sh        # creates .env and venv, installs dependencies
source .venv/bin/activate
./scripts/run_pipeline_local.sh    # ingest -> Bronze -> Silver -> Gold
./scripts/run_tests.sh             # run the test suite
```

The run prints the batch id and the locations of its outputs: `data/bronze/<batch_id>/`, `data/quarantine/<batch_id>/`, `data/silver/<batch_id>/` and `data/gold/<batch_id>/`.

A.2 Investigate a quarantined record

1. Open `data/quarantine/<batch_id>/`. Sub-folders are named by resource type; referential failures sit under `referential/<table>/`.
2. Each file is named after the record id and ends in `.error.json`.
3. Read `error_type` and `error_message` for the reason, `source_file` for where it came from, and `payload` for the record as received.
4. Fix the source data (or the rule, if the rule is wrong) and rerun; reruns are idempotent in cluster mode.

A.3 Docker Compose

```
cp .env.example .env
# edit .env: AIRFLOW_FERNET_KEY, AIRFLOW_DB_PASSWORD, AIRFLOW_ADMIN_PASSWORD,
#            DEID_SALT, GARAGE_RPC_SECRET, GARAGE_ADMIN_TOKEN
docker compose up -d
# Airflow UI  http://localhost:8080      Trino   http://localhost:8081
# Garage S3   http://localhost:3900      Nessie  http://localhost:19120
```

A.4 Kubernetes

```
kubectl apply -f k8s/namespace.yaml -f k8s/configmap.yaml
cp k8s/secret.example.yaml k8s/secret.local.yaml   # fill in real values
                                                   # never commit this file
kubectl apply -f k8s/secret.local.yaml
helm install garage ./charts/garage -f charts/garage/values.yaml \
  --set garage.rpcSecret=$(openssl rand -hex 32) \
  --set garage.adminToken=$(openssl rand -hex 32)
# then apply argocd/root-app.yaml after replacing the repository URL placeholder
```

A.5 Troubleshooting

Symptom	Likely cause and remedy
ModuleNotFoundError: No module named 'src'	Run from the repository root, or set PYTHONPATH to the repository root.
Spark job fails with a Py4J or JVM error	A JDK must be on the path; check with java -version.
ModuleNotFoundError: No module named 'distutils'	Python 3.12 or later with PySpark 3.5: install the pinned requirements, which include setuptools below 81.
pip install of PySpark succeeds but jobs fail strangely on Python 3.14	PySpark 3.5 does not support Python 3.14. Use Python 3.11 to 3.13, or move to PySpark 4.1 or later and retest.
pyarrow warnings during the Gold job	Harmless; install pyarrow 14 or later to enable faster pandas and Spark conversion.
apache-airflow install conflicts	Install it with the official constraints file for your Python version; do not run a bare pip install.
DAG run fails at the first task	That is usually the quality gate working as designed. Inspect the quarantine folder for the batch and read the error messages.
data_quality task reports no Silver records	Intentional: a gate that inspected nothing must not pass. Check the spark_transform logs and the Silver folder for the batch.
Compose refuses to start	A required variable is missing from .env; the error message names it.

Appendix B. Configuration reference

All settings are environment variables read by `src/utils/config.py`. Items marked sensitive must be supplied through Kubernetes Secrets or an untracked `.env` file and never committed.

Variable	Purpose	Default	Secret
`RUN_MODE`	local (Parquet, local filesystem) or cluster (Iceberg, S3)	local	No
`LOCAL_DATA_ROOT`	Root folder for layers in local mode	<repo>/data	No
`LOG_LEVEL`	Logging verbosity	INFO	No
`MAX_INVALID_RATIO`	Quality-gate threshold for the invalid share of a batch	0.2	No
`MEDICAL_PIPELINE_SCHEDULE`	Airflow schedule	@daily	No
`FHIR_SOURCE_DIR`	Folder of FHIR JSON files to ingest	data/sample/fhir	No
`DICOM_SOURCE_DIR`	Folder of DICOM metadata JSON	data/sample/dicom	No
`S3_ENDPOINT`	S3-compatible endpoint (Garage)	http://localhost:3900	No
`S3_REGION`	S3 region name	garage	No
`BUCKET_RAW / BUCKET_BRONZE / BUCKET_QUARANTINE`	Bucket names	medical-raw / medical-bronze / medical-quarantine	No
`WAREHOUSE_PATH`	Iceberg warehouse location	s3a://nessie-warehouse/	No
`NESSIE_URI`	Nessie API endpoint	http://localhost:19120/api/v2	No
`NESSIE_DEFAULT_BRANCH`	Nessie branch used by Spark and Trino	main	No
`ICEBERG_CATALOG_NAME`	Spark catalog name	iceberg	No
`SPARK_MASTER`	Spark master URL	local[*]	No
`S3_ACCESS_KEY / S3_SECRET_KEY`	Object storage credentials	empty	Yes
`DEID_SALT`	Salt for MRN hashing	development placeholder	Yes
`GARAGE_RPC_SECRET / GARAGE_ADMIN_TOKEN`	Garage cluster and admin secrets	none (required)	Yes
`AIRFLOW_FERNET_KEY / AIRFLOW_DB_PASSWORD / AIRFLOW_ADMIN_PASSWORD`	Airflow secrets (Compose)	none (required)	Yes

Appendix C. Repository map

```
medical-lakehouse-platform/
  dags/medical_data_pipeline.py   Airflow DAG: 7 tasks, retries, gates
  src/
    ingestion/                    FHIR and DICOM ingestion, quarantine writer
    fhir/                         FHIR R4 validator and parser
    deidentification/             Hash, remove and generalise PHI
    quality/                      Row-level checks, gate, Silver checks
    transformations/              Gold aggregation logic (pandas)
    storage/                      S3-compatible client, local fallback
    utils/                        Environment-driven config, JSON logging
  spark/jobs/                     bronze_to_silver, silver_to_gold, common
  sql/                            bronze, silver, gold DDL and analytics
  tests/                          80 tests: unit, integration, Spark, DAG
  data/sample/                    Synthetic FHIR bundles, DICOM metadata
  docker/, docker-compose.yml     Airflow image and local service stack
  charts/garage/                  Helm chart for Garage object storage
  k8s/                            Namespace, ConfigMap, Secret template, Job
  argocd/                         Root app and four component apps
  config/                         garage.toml, Trino catalog, Spark defaults
  scripts/                        init, run, test, backup, key-setup helpers
  docs/                           SYSTEM_DESIGN.md and four diagrams
  README.md  CHANGELOG.md  .env.example  requirements*.txt  pyproject.toml
```

Measure	Value
Pipeline Python (src, spark, dags)	about 2,470 lines
Test Python	about 1,000 lines across 11 test files
SQL	14 files, about 350 lines (DDL, example queries, restricted view)
Kubernetes, Helm and Argo CD manifests	19 YAML files plus the Compose file
Shell scripts	5
Diagrams	4, each in SVG and PNG

Appendix D. Glossary

Term	Meaning
FHIR	Fast Healthcare Interoperability Resources, an HL7 standard for exchanging clinical data as JSON resources such as Patient and Observation.
Bundle	A FHIR container that holds many resources in one document.
DICOM	Digital Imaging and Communications in Medicine, the standard for medical images and their metadata.
SOP Instance UID	The unique identifier of one DICOM object; used as the key of dicom_metadata.
PHI	Protected health information: health data that can identify an individual.
MRN	Medical record number, a hospital's identifier for a patient.
LOINC	A coding system for laboratory and clinical observations.
De-identification	Removing or transforming identifying information so records cannot easily be tied to a person.
Pseudonymisation	Replacing identifiers with a stable substitute so records can still be linked internally; weaker than full anonymisation.
Safe Harbor	A HIPAA method that lists identifiers to remove or generalise for data to be considered de-identified.
Medallion architecture	Refining data through Bronze (raw), Silver (clean) and Gold (aggregated) layers.
Lakehouse	An architecture that combines the low-cost storage of a data lake with the table features of a warehouse.
Apache Iceberg	An open table format giving ACID transactions, schema evolution and snapshots over Parquet files.
Project Nessie	A catalog that versions table state like Git, with branches and commits.
Trino	A distributed SQL query engine that queries data where it lives.
Garage	A lightweight, self-hosted, S3-compatible object store.
DAG	Directed acyclic graph; in Airflow, a pipeline of tasks with dependencies.
XCom	Airflow's mechanism for passing small values between tasks.
Quarantine	The holding area for rejected records and the reason for each rejection.
Referential integrity	The rule that a reference (such as patient_id) must point to an existing parent record.
Quality gate	A threshold check that halts the pipeline when too much of a batch is invalid.
Idempotent	Safe to run more than once with the same result.
MERGE INTO	A SQL statement that updates matching rows and inserts new ones in a single operation.
GitOps	Managing infrastructure by declaring it in Git and letting an agent keep the system matched to it.
Argo CD	A GitOps controller for Kubernetes.
Helm	A package manager for Kubernetes applications.
HMAC	A keyed hash; unlike a salted hash, it cannot be recomputed without the secret key.
RPO / RTO	Recovery point objective (acceptable data loss) and recovery time objective (acceptable downtime).